

INDEX OF PRACTICALS

Object Oriented Programming Lab (C++)

S.No.	Problem Title / Practical Description	Page	Remarks
1	Function Overloading for Volume Calculation (Cube, Cylinder, Cuboid)	2	
2	Class Representation of Bank Account Management	3	
3	Addition of Time Objects by Passing and Returning Objects	4	
4	Currency Comparison (USD & INR) using Friend Function	5	
5	Complex Numbers with Constructor Overloading	6	
6	Static Data Members and Member Functions for Object Tracking	7	
7	Menu-Driven Shopping Cart with Static Data Members & Switch Cases	8	
8	Dynamic Memory Allocation for Array of Objects using 'new' and 'delete'	11	
9	Object Comparison and Returning Invoking Object using 'this' Pointer	12	
10	Unary Operator Overloading (Negation '-' on Complex Number)	13	
11	Prefix and Postfix Unary Operators (++ and --) Overloading	14	
12	Stream Insertion (<<) and Extraction (>>) Operator Overloading	15	
13	Four-Way User-Defined Data Type Conversions	16	

Practical / Question No. 1

Function Overloading for Volume Calculation (Cube, Cylinder, Cuboid)

Question: Write a C++ program to calculate the volume of different geometric shapes (Cube, Cylinder, and Cuboid) using function overloading.

Source Code:

```
// Question: Write a C++ program to calculate the volume of different geometric shapes
// (Cube, Cylinder, and Cuboid) using function overloading.

#include <iostream>
using namespace std;

float volume(float s) {
    return s * s * s;
}

float volume(float r, float h) {
    return 3.14 * r * r * h;
}

float volume(float l, float b, float h) {
    return l * b * h;
}

int main() {
    float s, r, h, l, b;
    cout << "Enter side of cube: ";
    cin >> s;
    cout << "Volume of cube: " << volume(s) << endl;

    cout << "Enter radius and height of cylinder: ";
    cin >> r >> h;
    cout << "Volume of cylinder: " << volume(r, h) << endl;

    cout << "Enter length, breadth and height of cuboid: ";
    cin >> l >> b >> h;
    cout << "Volume of cuboid: " << volume(l, b, h) << endl;

    return 0;
}
```

Example Output:

```
$ ./01-FunctionOverloading
Enter side of cube: 5
Volume of cube: 125
Enter radius and height of cylinder: 3 7
Volume of cylinder: 197.82
Enter length, breadth and height of cuboid: 2 3 4
Volume of cuboid: 24
```

Practical / Question No. 2

Class Representation of Bank Account Management

Question: Define a class to represent a bank account.

Source Code:

```
// Question: Define a class to represent a bank account.

#include <iostream>
#include <string>

using namespace std;

class BankAccount {
    string name;
    int accNum;
    double balance;

public:
    void init(string n, int a, double b) {
        name = n;
        accNum = a;
        balance = b;
    }

    void deposit(double amount) {
        cout << "Depositing: " << amount << endl;
        balance += amount;
    }

    void withdraw(double amount) {
        cout << "Withdrawing: " << amount << endl;
        if (amount > balance) {
            cout << "Insufficient balance\n";
        } else {
            balance -= amount;
        }
    }

    void display() {
        cout << "Name: " << name << "\nBalance: " << balance << "\n";
    }
};

int main() {
    BankAccount acc;
    acc.init("Additya", 101, 5000);
    acc.deposit(1500);
    acc.display();
    acc.withdraw(2000);
    acc.display();
    return 0;
}
```

Example Output:

```
$ ./02-BankAccount
Depositing: 1500
Name: Additya
Balance: 6500
Withdrawing: 2000
Name: Additya
Balance: 4500
```

Practical / Question No. 3

Addition of Time Objects by Passing and Returning Objects

Question: Create a class Time to store time in hours and minutes and a function to add two Time objects.

Source Code:

```
// Question: Create a class Time to store time in hours and minutes and a function to add two Time objects.

#include <iostream>
using namespace std;

class Time {
    int hours, minutes;
public:
    Time(int h = 0, int m = 0) : hours(h), minutes(m) {}

    void getTime() {
        cout << "Enter hours and minutes: ";
        cin >> hours >> minutes;
    }

    Time sum(Time t2) {
        Time t;
        t.minutes = minutes + t2.minutes;
        t.hours = hours + t2.hours + t.minutes / 60;
        t.minutes %= 60;
        return t;
    }

    void display() {
        cout << hours << " hrs " << minutes << " mins\n";
    }
};

int main() {
    Time t1, t2, t3;
    t1.getTime();
    t2.getTime();

    t3 = t1.sum(t2);

    cout << "Time 1: ";
    t1.display();
    cout << "Time 2: ";
    t2.display();
    cout << "Sum: ";
    t3.display();

    return 0;
}
```

Example Output:

```
$ ./03-TimeAddition
Enter hours and minutes: 2 45
Enter hours and minutes: 3 30
Time 1: 2 hrs 45 mins
Time 2: 3 hrs 30 mins
Sum: 6 hrs 15 mins
```

Practical / Question No. 4

Currency Comparison (USD & INR) using Friend Function

Question: Write a C++ program with USD and INR classes to find the greater currency value using a friend function (1 USD = 95 INR).

Source Code:

```
// Question: Write a C++ program with USD and INR classes to find the greater currency
// value using a friend function (1 USD = 95 INR).

#include <iostream>
using namespace std;

class INR;

class USD {
    float amount;

public:
    USD(float a = 0) : amount(a) {}

    void display() {
        cout << "$" << amount << endl;
    }

    friend void findMax(USD, INR);
};

class INR {
    float amount;

public:
    INR(float a = 0) : amount(a) {}

    void display() {
        cout << "Rs. " << amount << endl;
    }

    friend void findMax(USD, INR);
};

void findMax(USD u, INR i) {
    float usdInInr = u.amount * 95;
    if (usdInInr > i.amount) {
        cout << "$" << u.amount << " is greater than Rs. " << i.amount << endl;
    } else if (i.amount > usdInInr) {
        cout << "Rs. " << i.amount << " is greater than $" << u.amount << endl;
    } else {
        cout << "Both amounts are equal" << endl;
    }
}

int main() {
    USD u(50);
    INR i(4500);

    findMax(u, i);
    return 0;
}
```

Example Output:

```
$ ./04-CurrencyFriendFunction
$50 is greater than Rs. 4500
```

Practical / Question No. 5

Complex Numbers with Constructor Overloading

Question: Write a C++ program to create a Complex class representing complex numbers. Demonstrate the use of default constructor, parameterized constructor and copy constructor

Source Code:

```
// Question: Write a C++ program to create a Complex class representing complex numbers.
// Demonstrate the use of default constructor, parameterized constructor and copy constructor

#include <iostream>
using namespace std;

class Complex {
    float real, imag;
public:
    Complex() : real(0), imag(0) {
        cout << real << " + " << imag << "i : created by Default constructor\n";
    }

    Complex(float r, float i) : real(r), imag(i) {
        cout << real << " + " << imag << "i : created by Parameterized constructor\n";
    }

    Complex(const Complex& c) : real(c.real), imag(c.imag) {
        cout << real << " + " << imag << "i : created by Copy constructor\n";
    }
};

int main() {
    Complex c1;
    Complex c2(3, 4);
    Complex c3 = c2;
    return 0;
}
```

Example Output:

```
$ ./05-ComplexNumbersConstructors
0 + 0i : created by Default constructor
3 + 4i : created by Parameterized constructor
3 + 4i : created by Copy constructor
```

Practical / Question No. 6

Static Data Members and Member Functions for Object Tracking

Question: Write a C++ program to demonstrate the use of static data members and static member functions to track the number of current students.

Source Code:

```
// Question: Write a C++ program to demonstrate the use of static data members and static
// member functions to track the number of current students.

#include <iostream>
#include <string>
using namespace std;

class Student {
    int rollNo;
    string name;
    static int count;

public:
    Student(int r, string n) : rollNo(r), name(n) {
        count++;
    }

    ~Student() {
        count--;
    }

    void display() {
        cout << "Roll no." << rollNo << " : " << name << endl;
    }

    static int getCount() {
        return count;
    }
};

int Student::count = 0;

int main() {
    Student s1(1, "Ayush");
    Student s2(2, "Bipin");

    s1.display();
    s2.display();
    cout << "Students: " << Student::getCount() << endl;

    {
        Student s3(3, "Gaurav");
        s3.display();
        cout << "Students: " << Student::getCount() << endl;
    }

    cout << "Left the Gaurav block." << endl;

    cout << "Students: " << Student::getCount() << endl;
    return 0;
}
```

Example Output:

```
$ ./06-StaticMembers
Roll no.1 : Ayush
Roll no.2 : Bipin
Students: 2
Roll no.3 : Gaurav
Students: 3
Left the Gaurav block.
Students: 2
```

Practical / Question No. 7

Menu-Driven Shopping Cart with Static Data Members & Switch Cases

Question: Write a C++ program to implement a Shopping Cart system using classes, static variables, and switch cases.

Source Code:

```
// Question: Write a C++ program to implement a Shopping Cart system using classes,
// static variables, and switch cases.

#include <iostream>
#include <string>
using namespace std;

class Item {
    int itemCode;
    string name;
    float price;
    int cartQty;

public:
    static float totalPrice;

    void addItem() {
        cout << "Enter item code, name, and price: ";
        cin >> itemCode >> name >> price;
        cartQty = 0;
    }

    void addToCart(int qty) {
        cartQty += qty;
        totalPrice += price * qty;
    }

    void removeFromCart(int qty) {
        if (qty > cartQty) qty = cartQty;
        totalPrice -= price * qty;
        cartQty -= qty;
    }

    void showItem() {
        if (cartQty > 0) {
            cout << itemCode << "\t" << name << "\t" << price << "\t" << cartQty << "\t" << price * cartQty
<< "\n";
        }
    }

    void clear() {
        totalPrice -= price * cartQty;
        cartQty = 0;
    }

    int getCode() {
        return itemCode;
    }

    static void showBill() {
        cout << "Total Bill: " << totalPrice << "\n";
    }

    static void clearBill() {
        totalPrice = 0;
    }
};

float Item::totalPrice = 0;

int main() {
    Item items[50];
    int count = 0;
    int choice;

    do {
        cout << "\n0. Initialise cart\n1. Add Item\n2. Remove Item\n3. Show Bill\n4. Clear Cart\n5.
Exit\nEnter choice: ";
        cin >> choice;

        switch (choice) {
            case 0:
                for (int i = 0; i < count; i++) {
```

```

        items[i].clear();
    }
    Item::clearBill();
    count = 0;
    cout << "Cart initialised.\n";
    break;

case 1: {
    int qty;
    items[count].addItem();
    cout << "Enter quantity: ";
    cin >> qty;
    items[count].addToCart(qty);
    count++;
    break;
}

case 2: {
    int code, qty;
    cout << "Enter item code to remove: ";
    cin >> code;
    bool found = false;
    for (int i = 0; i < count; i++) {
        if (items[i].getCode() == code) {
            cout << "Enter quantity to remove: ";
            cin >> qty;
            items[i].removeFromCart(qty);
            found = true;
            break;
        }
    }
    if (!found) cout << "Item not found in cart.\n";
    break;
}

case 3:
    cout << "Code\tName\tPrice\tQty\tTotal\n";
    for (int i = 0; i < count; i++) {
        items[i].showItem();
    }
    Item::showBill();
    break;

case 4:
    for (int i = 0; i < count; i++) {
        items[i].clear();
    }
    Item::clearBill();
    count = 0;
    cout << "Cart cleared.\n";
    break;

case 5:
    break;

default:
    cout << "Invalid choice.\n";
}
} while (choice != 5);

return 0;
}

```

Example Output:

```
$ ./07-ShoppingCart

0. Initialise cart
1. Add Item
2. Remove Item
3. Show Bill
4. Clear Cart
5. Exit
Enter choice: 1
Enter item code, name, and price: 101 Pen 15
Enter quantity: 2

0. Initialise cart
1. Add Item
2. Remove Item
3. Show Bill
4. Clear Cart
5. Exit
Enter choice: 1
Enter item code, name, and price: 102 Notebook 50
Enter quantity: 3

0. Initialise cart
1. Add Item
2. Remove Item
3. Show Bill
4. Clear Cart
5. Exit
Enter choice: 3
Code Name Price Qty Total
101 Pen 15 2 30
102 Notebook 50 3 150
Total Bill: 180

0. Initialise cart
1. Add Item
2. Remove Item
3. Show Bill
4. Clear Cart
5. Exit
Enter choice: 2
Enter item code to remove: 101
Enter quantity to remove: 1

0. Initialise cart
1. Add Item
2. Remove Item
3. Show Bill
4. Clear Cart
5. Exit
Enter choice: 3
Code Name Price Qty Total
101 Pen 15 1 15
102 Notebook 50 3 150
Total Bill: 165

0. Initialise cart
1. Add Item
2. Remove Item
3. Show Bill
4. Clear Cart
5. Exit
Enter choice: 5
```

Practical / Question No. 8

Dynamic Memory Allocation for Array of Objects using 'new' and 'delete'

Question: Write a C++ program to demonstrate dynamic memory allocation using 'new' and 'delete' operators for an array of complex numbers.

Source Code:

```
// Question: Write a C++ program to demonstrate dynamic memory allocation using 'new' and
// 'delete' operators for an array of complex numbers.

#include <iostream>
using namespace std;

class Complex {
    float real, imag;

public:
    void get() {
        cin >> real >> imag;
    }

    void display() {
        cout << real << " + " << imag << "i\n";
    }
};

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    Complex* arr = new Complex[n];

    for (int i = 0; i < n; i++) {
        cout << "Enter real and imag for number " << i + 1 << ": ";
        arr[i].get();
    }

    cout << "\nComplex Numbers:\n";
    for (int i = 0; i < n; i++) {
        arr[i].display();
    }

    delete[] arr;
    return 0;
}
```

Example Output:

```
$ ./08-DynamicArrayAllocation
Enter number of elements: 2
Enter real and imag for number 1: 3 4
Enter real and imag for number 2: 5 -2

Complex Numbers:
3 + 4i
5 + -2i
```

Practical / Question No. 9

Object Comparison and Returning Invoking Object using 'this' Pointer

Question: Write a C++ program to compare two persons by age and return the older person using the 'this' pointer.

Source Code:

```
// Question: Write a C++ program to compare two persons by age and return the older person using the 'this' pointer.

#include <iostream>
#include <string>
using namespace std;

class Person {
    string name;
    int age;

public:
    Person(string name, int age) {
        this->name = name;
        this->age = age;
    }

    Person& older(Person& p) {
        if (p.age > this->age) return p;
        return *this;
    }

    void display() {
        cout << name << " (" << age << ")\n";
    }
};

int main() {
    Person p1("Alice", 21), p2("Bob", 25);
    Person res = p1.older(p2);
    cout << "Older: ";
    res.display();
    return 0;
}
```

Example Output:

```
$ ./09-ThisPointerComparison
Older: Bob (25)
```

Practical / Question No. 10

Unary Operator Overloading (Negation '-' on Complex Number)

Question: Write a C++ program to overload unary minus (-) operator for a Complex class.

Source Code:

```
// Question: Write a C++ program to overload unary minus (-) operator for a Complex class.

#include <iostream>
using namespace std;

class Complex {
    float real, imag;
public:
    Complex(float r = 0, float i = 0) : real(r), imag(i) {}

    Complex operator-() {
        return Complex(-real, -imag);
    }

    void display() {
        cout << real << " + " << imag << "i\n";
    }
};

int main() {
    Complex c(3, -4);
    cout << "Original: ";
    c.display();

    Complex neg = -c;
    cout << "Negated: ";
    neg.display();

    return 0;
}
```

Example Output:

```
$ ./10-UnaryOperatorOverloading
Original: 3 + -4i
Negated: -3 + 4i
```

Practical / Question No. 11

Prefix and Postfix Unary Operators (++ and --) Overloading

Question: Write a C++ program to overload unary increment (++) and decrement (--) operators in both prefix and postfix forms for a Page class.

Source Code:

```
// Question: Write a C++ program to overload unary increment (++) and decrement (--)
// operators in both prefix and postfix forms for a Page class.

#include <iostream>
using namespace std;

class Page {
    int pageNo;

public:
    Page(int p = 1) : pageNo(p) {}

    Page& operator++() {
        ++pageNo;
        return *this;
    }

    Page operator++(int) {
        Page temp = *this;
        pageNo++;
        return temp;
    }

    Page& operator--() {
        --pageNo;
        return *this;
    }

    Page operator--(int) {
        Page temp = *this;
        pageNo--;
        return temp;
    }

    void display() {
        cout << "Page: " << pageNo << endl;
    }
};

int main() {
    Page p(10);

    ++p;
    cout << "Prefix ++: ";
    p.display();

    p++;
    cout << "Postfix ++: ";
    p.display();

    --p;
    cout << "Prefix --: ";
    p.display();

    p--;
    cout << "Postfix --: ";
    p.display();

    return 0;
}
```

Example Output:

```
$ ./11-PrefixPostfixOperators
Prefix ++: Page: 11
Postfix ++: Page: 12
Prefix --: Page: 11
Postfix --: Page: 10
```

Practical / Question No. 12

Stream Insertion (<<) and Extraction (>>) Operator Overloading

Question: Write a C++ program to overload stream insertion (<<) and stream extraction (>>) operators to read and display user-defined Time objects.

Source Code:

```
// Question: Write a C++ program to overload stream insertion (<<) and stream extraction (>>)
// operators to read and display user-defined Time objects.

#include <iostream>
using namespace std;

class Time {
    int hours, minutes, seconds;

public:
    Time(int h = 0, int m = 0, int s = 0) : hours(h), minutes(m), seconds(s) {}

    friend istream& operator>>(istream& in, Time& t) {
        cout << "Enter hours, minutes, and seconds: ";
        in >> t.hours >> t.minutes >> t.seconds;
        return in;
    }

    friend ostream& operator<<(ostream& out, const Time& t) {
        out << t.hours << "h " << t.minutes << "m " << t.seconds << "s";
        return out;
    }
};

int main() {
    Time t;
    cin >> t;
    cout << "Time: " << t << endl;
    return 0;
}
```

Example Output:

```
$ ./12-StreamOperatorOverloading
Enter hours, minutes, and seconds: 10 25 45
Time: 10h 25m 45s
```

Practical / Question No. 13

Four-Way User-Defined Data Type Conversions

Question: Write a C++ program to demonstrate type conversions

Source Code:

```
// Question: Write a C++ program to demonstrate type conversions

#include <iostream>
using namespace std;

class Minutes {
    int totalMins;

public:
    // Predefined to User-defined (via constructor)
    Minutes(int m = 0) : totalMins(m) {}

    // User-defined to Predefined (via operator overloading)
    operator int() {
        return totalMins;
    }

    int getMins() {
        return totalMins;
    }

    void display() {
        cout << totalMins << " mins\n";
    }
};

class HoursMinutes {
    int hr, min;

public:
    HoursMinutes(int h = 0, int m = 0) : hr(h), min(m) {}

    // User-defined to User-defined (via constructor)
    HoursMinutes(Minutes m) {
        hr = m.getMins() / 60;
        min = m.getMins() % 60;
    }

    // User-defined to User-defined (via operator overloading)
    operator Minutes() {
        return Minutes(hr * 60 + min);
    }

    void display() {
        cout << hr << " hrs " << min << " mins\n";
    }
};

int main() {
    Minutes m1 = 135;
    cout << "Predefined to User-defined (Constructor): ";
    m1.display();

    int rawMins = m1;
    cout << "User-defined to Predefined (Operator): " << rawMins << endl;

    HoursMinutes hm1 = m1;
    cout << "User-defined to User-defined (Constructor): ";
    hm1.display();

    HoursMinutes hm2(2, 30);
    Minutes m2 = hm2;
    cout << "User-defined to User-defined (Operator): ";
    m2.display();

    return 0;
}
```

Example Output:

```
$ ./13-TypeConversion
Predefined to User-defined (Constructor): 135 mins
User-defined to Predefined (Operator): 135
User-defined to User-defined (Constructor): 2 hrs 15 mins
User-defined to User-defined (Operator): 150 mins
```